AI Invoice Auditor

A Containerized, Multilingual Invoice Validation System

1. Project Overview

Business Context

A global logistics firm receives hundreds of vendor invoices daily from suppliers around the world, in different formats (PDF, DOCX, scanned images) and different languages (English, Spanish, German and others). Invoices are currently reviewed by hand and checked against internal ERP records to find discrepancies, missing details or fraudulent submissions.

That manual process is slow, especially with multilingual documents; error-prone, because it depends on human attention across hundreds of line items; and expensive, because it requires skilled auditors who can also read foreign languages.

Project Objective

Design, build and deploy an end-to-end agentic AI system, running as a containerized multi-service application, that:

- Ingests multilingual invoices from a monitored folder.
- Extracts structured data from PDF, DOCX and scanned image formats.
- Translates non-English content into English.
- Validates the extracted data against an ERP service backed by PostgreSQL.
- Produces discrepancy reports and stores them in a durable audit trail.
- Answers natural-language questions about processed invoices using a RAG pipeline.
- Allows a human auditor to review, correct and re-validate through a web dashboard.
- Starts completely, from a clean machine, with a single command.

What makes this capstone different: The system is not considered complete when it runs on your laptop. It is complete when the entire stack can be destroyed and recreated, and the audit history and RAG index are still intact. Persistence and deployment are assessed, not assumed.

2. Problem Statement

Build an agentic system that monitors an incoming folder, processes each invoice through a LangGraph workflow, and produces a validated, auditable outcome. The system must:

- Detect and route new invoice files as they arrive.
- Extract invoice number, date, vendor, currency, totals and line items from any supported format.
- Detect the source language and translate to English where required.

- Validate completeness and internal arithmetic against configurable rules.
- Validate quantity, unit price and currency against ERP master data.
- Flag discrepancies beyond a configured tolerance and recommend approve, manual review or reject.
- Persist every decision, with its evidence, so the audit trail survives infrastructure changes.
- Index processed invoices and reports so they can be queried in natural language.
- Accept human corrections and re-run validation on demand.

3. Functional Requirements

3.1 Invoice Monitoring

- Watch the incoming directory, which is mounted into the agent container as a volume.
- Detect new files and route them by type to the correct extraction path.
- Compute a checksum for each file and skip any invoice already present in the audit trail.
- Record every detected file, including ones that fail to process.

Why a checksum rather than a filename: Vendors rename files freely, and the same invoice can arrive twice under different names. A content checksum is the only reliable way to answer whether this invoice has already been audited.

3.2 Invoice Extraction

- Handle `.pdf`, `.docx` and `.png` (scanned) input.
- For PDFs, attempt direct text extraction first; fall back to OCR when no embedded text layer exists.
- Use Tesseract with the language packs required by the dataset for scanned documents.
- Extract invoice number, date, vendor name, currency and total.
- Extract line items: item code, description, quantity, unit price and line total.
- Store the complete raw extracted text, not only the parsed fields.

Store the raw text: OCR is expensive and cannot be cheaply repeated. Keeping the full extracted text means the vector index can be rebuilt without re-reading a single PDF, and a disputed audit can be defended by showing exactly what the system read.

3.3 Translation

- Detect the source language automatically.
- Translate non-English content to English using AWS Bedrock models.
- Record a translation confidence score alongside the translated text.
- Cache translation results so repeated content is not re-translated.

3.4 Validation

Invoice Data Validation Agent

- Check field completeness against `configs/rules.yaml`.
- Verify that line totals sum to the invoice total and that tax calculations are correct.
- Highlight missing or internally inconsistent fields.

Business Validation Agent

Calls the mock ERP service over HTTP by service name. The ERP service reads its data from PostgreSQL:

- `GET http://mock-erp:8000/po/{po_number}`: PO header and line items
- `GET http://mock-erp:8000/vendor/{vendor_id}`: Vendor master record
- `GET http://mock-erp:8000/sku/{sku_code}`: SKU master record
- Compare quantity, unit price and currency against the matching PO.
- Apply the tolerance thresholds defined in `rules.yaml`.
- Record one discrepancy row per mismatched field, with both values and the deviation.

Service boundary – this is assessed: The agent container can technically reach PostgreSQL directly, because services share the same database. It must not query ERP tables that way. All ERP data is obtained through the mock-erp HTTP API. Bypassing the API collapses the service boundary and makes the ERP service meaningless.

3.5 Reporting

- Generate a JSON report for machine consumption and an HTML report for human reading.
- Include a discrepancy summary, missing field list and translation confidence.
- Produce a recommendation: `approve`, `manual review`, or `reject`.
- Write the outcome to PostgreSQL. Report files may also be written, but the database is the system of record.

3.6 RAG-based Querying

Implement a LangGraph-based RAG pipeline over the processed invoices and their reports:

- **Indexing Agent:** Chunk invoice text and report text, embed, and write to Qdrant.
- **Retrieval Agent:** Retrieve relevant chunks, with the ability to filter to a specific invoice.
- **Augmentation Agent:** Rerank and assemble context.
- **Generation Agent:** Answer the question through LiteLLM.
- **Reflection Agent:** Evaluate the answer using the RAG Triad (relevance, groundedness, context relevance).
- If the overall avg score is greater than 0.7 then generated answer should be displayed to the user else retrieve the chunks to generate response for the user query (max retries: 3).

3.7 Human-in-the-Loop Dashboard

A Streamlit interface, running in its own container, that allows an auditor to:

- Review invoice reports and their discrepancies.
- Correct extracted values.
- Ask natural-language questions about processed invoices.
- Record feedback, which is stored permanently rather than overwriting the original extraction.

4. Non-Functional Requirements

Requirement	Expectation
Performance	Efficient processing of invoice batches; expensive operations such as OCR and translation are performed once and reused.
Scalability	Services are independently deployable. Be prepared to explain which service could be scaled horizontally and which could not.
Durability	The audit trail and RAG index survive destruction and recreation of every container.
Security	No credential appears in application source, in any Dockerfile, or in any built image layer.
Reliability	A service that cannot reach its dependencies fails clearly and immediately rather than starting in a broken state.
Usability	The dashboard is usable by a non-technical auditor.
Maintainability	Modular agent design, documented configuration, reproducible build.

5. Solution Architecture

The system is deployed as six containers on a single user-defined bridge network created by Docker Compose. Services address each other by service name; no IP address is ever hard-coded.

Service	Image/Build	Ports (host:container)	Responsibility
`agent`	Build from Dockerfile	`8000:8000`	Runs the LangGraph workflow: monitor, extractor, translation, validation, reporting and RAG agents. Exposes a health endpoint.
`mock-erp`	Build from Dockerfile	`8001:8000`	FastAPI service serving vendor, purchase order and SKU lookups from PostgreSQL.
`streamlit-ui`	Build from Dockerfile	`8501:8501`	Human-in-the-loop dashboard. Reads reports from PostgreSQL and calls the agent for re-validation.
`postgres`	`postgres:16`	`5432:5432`	Durable system of record. Holds agent checkpoints, ERP master data and the audit trail.

Service	Image/Build	Ports (host:container)	Responsibility
qdrant	qdrant/qdrant	6333:6333	Vector database holding the RAG index. A derived store, rebuildable from PostgreSQL.
redis	redis:7	6379:6379	Ephemeral cache for translation results and LLM responses.

5.1 Processing Flow

A single invoice moves through the system as follows:

1. The file arrives in the mounted incoming directory. The file itself never enters any database.
2. The monitor agent detects it, checksums it, and routes it by type.
3. The extractor produces raw text — directly for digital PDFs, via OCR for scans — and writes that text to PostgreSQL.
4. If same query is asked instead of hitting the LLM, same result for query is taken from redis.
5. Structured fields are parsed into typed columns; line items become their own rows.
6. The validation agent calls `mock-erp` over HTTP and writes discrepancies to PostgreSQL.
7. The reporting agent writes the recommendation to PostgreSQL and generates report files.
8. The indexing agent chunks the stored text, embeds it, and writes vectors to Qdrant.

Queries take a much shorter path: Structured questions are answered directly from PostgreSQL with SQL. Only open-ended questions go through Qdrant retrieval and generation. Nothing is re-extracted at query time.

Note on LiteLLM: LiteLLM runs as an in-process library in this build rather than a separate container. In production it would run as its own gateway service so that rate limiting, cost tracking and credential handling are centralised.

Architectural Assessment Highlights

- The agent is the only service that writes application data. `mock-erp` reads the `erp` schema and serves it over HTTP; `streamlit-ui` reads the `audit` schema and calls the agent when something must change. Keeping writes in one place is what makes the audit trail trustworthy.
- `mock-erp` is the only route to ERP data. Querying the `erp` schema from outside `mock-erp` collapses the service boundary.
- Qdrant points back to PostgreSQL, not the other way round. The vector index is derived from text already stored in PostgreSQL, so losing the Qdrant volume is a rebuild rather than data loss.
- **Host Port vs Container Port:** `mock-erp` is published on host port 8001 but addressed internally as `mock-erp:8000`.

6. Data Architecture

Store	Holds	Survives teardown?
Filesystem (volume)	The original invoice files and generated report files.	Yes, while the volume exists.
PostgreSQL	Agent checkpoints, ERP master data, the complete audit trail.	Yes — named volume required.
Qdrant	Vector embeddings and their chunk payloads.	Yes — named volume required.
Redis	LLM response caches.	No, and it must not need to.

6.1 Where the Invoice File Itself Goes

The PDF, DOCX or PNG binary is never stored in PostgreSQL. Binary blobs in a relational database are an antipattern; production systems place them in object storage and keep a reference in the database. Here the file stays on the mounted volume and the database stores its path and checksum.

Text goes into PostgreSQL in three forms:

- **Unstructured text (`TEXT`):** The complete extracted text of the invoice as a single value, newlines and all.
- **Structured fields (Typed columns):** `invoice_number VARCHAR`, `invoice_date DATE`, `total_amount NUMERIC(14,2)` for filtering, sorting and summing.
- **Variable extras (`JSONB`):** Format-specific fields that do not justify a column of their own.

6.2 PostgreSQL: Three Schemas, One Database

Schema	Owner	Contents	Who designs it
`langgraph`	`agent`	checkpoints, checkpoint_blobs, checkpoint_writes, threads.	Created automatically by LangGraph PostgreSQL checkpointer.
`erp`	`mock-erp`	vendors, purchase_orders, po_line_items, sku_master.	Provided in `erp_schema.sql`.
`audit`	`agent` writes, `streamlit-ui` reads	invoice_audit, invoice_line_items, discrepancies, human_feedback.	You design these tables.

Suggested Audit Schema

SQL

```
CREATE SCHEMA IF NOT EXISTS audit;

CREATE TABLE audit.invoice_audit (
    invoice_id UUID PRIMARY KEY,
    file_path VARCHAR(512),
    file_checksum VARCHAR(64) UNIQUE,     -- detects reprocessing
    extraction_method VARCHAR(32),        -- pdfplumber | docx | tesseract
    raw_extracted_text TEXT,              -- full text, pre-translation
    translated_text TEXT,
    source_language VARCHAR(8),
    translation_confidence NUMERIC(4,3),
    vendor_name VARCHAR(255),
    po_number VARCHAR(64),
    invoice_date DATE,
    currency CHAR(3),
    total_amount NUMERIC(14,2),
    validation_status VARCHAR(32),        -- passed | failed | partial
    recommendation VARCHAR(32),           -- approve | review | reject
    extracted_raw JSONB,                  -- variable leftovers
    thread_id VARCHAR(128),               -- links to langgraph state
    processed_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit.invoice_line_items (
    id BIGSERIAL PRIMARY KEY,
    invoice_id UUID REFERENCES audit.invoice_audit(invoice_id),
    line_number INTEGER,
    item_code VARCHAR(64),
    description TEXT,
    quantity NUMERIC(12,3),
    unit_price NUMERIC(14,4),
    line_total NUMERIC(14,2)
);

CREATE TABLE audit.discrepancies (
    id BIGSERIAL PRIMARY KEY,
    invoice_id UUID REFERENCES audit.invoice_audit(invoice_id),
    field_name VARCHAR(64),
    invoice_value TEXT,
    erp_value TEXT,
    deviation_pct NUMERIC(8,4),
    severity VARCHAR(16),
    detected_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit.human_feedback (
    id BIGSERIAL PRIMARY KEY,
    invoice_id UUID REFERENCES audit.invoice_audit(invoice_id),
    field_name VARCHAR(64),
    original_value TEXT,
    corrected_value TEXT,
    corrected_by VARCHAR(128),
    corrected_at TIMESTAMPTZ DEFAULT now(),
    revalidated BOOLEAN DEFAULT FALSE
);
```

6.3 Qdrant: A Derived Index

Store a payload containing at minimum:

- `invoice_id`: Allows retrieval to be filtered to one invoice.
- `chunk_text`: The text the vector was produced from, returned to generation as context.
- `source_type`: Distinguishes invoice text from report text.
- `vendor`: Supports vendor-scoped questions.
- `language`: Supports filtering and diagnosis.

6.4 Redis: Disposable by Design

Cache only values that can be recomputed, each with an expiry:

- Key format: `llm:(sha256(prompt)) -> cached model response`

6.5 Service Addresses Inside the Network

- `agent` → `postgres`: `postgres:5432`
- `agent` → `qdrant`: `qdrant:6333`
- `agent` → `redis`: `redis:6379`
- `agent` → `mock-erp`: `mock-erp:8000`
- `streamlit-ui` → `agent`: `agent:8000`
- Browser → `streamlit-ui`: `localhost:8501`

7. Implementation Details

7.1 Technology Stack

- **Agent framework:** LangGraph, LangChain
- **LLM gateway:** LiteLLM (in-process)
- **Models:** AWS Bedrock (Nova Lite, Cohere Command R+)
- **API services:** FastAPI, Uvicorn, Pydantic
- **OCR and parsing:** Tesseract, Poppler, pdfplumber, python-docx
- **Vector DB:** Qdrant
- **Relational DB:** PostgreSQL 16
- **Cache:** Redis 7
- **UI:** Streamlit
- **Containerization:** Docker, Docker Compose v2
- **Base image:** `python:3.11-slim` (pinned)
- **Config:** YAML persona and rules files

7.2 Model Selection

Task	Model	Reason
Reasoning & summarization	Nova Lite	Efficient for structured reasoning and report generation.
RAG generation	Cohere Command R+	Optimised for retrieval-augmented generation.
Translation	Cohere Command R+	Handles multilingual translation natively.

7.3 Agent Specification

Agent	Function	Uses LLM
Invoice Monitor	Detects and routes new files; checksums against audit trail.	No
Extractor	Parses PDF, DOCX and images; falls back to OCR when needed.	No
Translation	Detects language, translates to English, caches results.	Yes
Invoice Data Validation	Checks completeness, totals and tax arithmetic against `rules.yaml`.	Yes
Business Validation	Compares invoice data to ERP records via `mock-erp` API.	Yes
Reporting	Produces discrepancy report and recommendation; persists outcome.	Yes
RAG agents	Indexing, retrieval, augmentation, generation and reflection.	Yes

8. Containerization Requirements

8.1 Common Dockerfile Requirements

- Base image pinned to an explicit version (no `latest` tag).
- Layer ordering optimized for caching.
- `.dockerignore` present excluding `.env`, `__pycache__`, `.git/`, and venvs.
- No credentials or API keys in any layer (checked via `docker history`).
- Process runs in foreground under an unprivileged non-root user.
- Logs written to `stdout`/`stderr`.
- Images tagged explicitly (e.g., `my-agent:1.0`).

8.2 Service Specifications

- **`agent`**: Installs OS binaries (`tesseract-ocr` + language packs, `poppler-utils`), Python libs, size ≤1.5 GB, health endpoint on port 8000.
- **`mock-erp`**: Slim Python base, FastAPI on container port 8000 published on host 8001.
- **`streamlit-ui`**: Slim Python base, binds to `0.0.0.0:8501`.

8.3 Compose Requirements

- Single command startup: `docker compose up --build -d`.
- Named volumes for `postgres` and `qdrant`.
- Healthchecks on databases; `depends_on: condition: service_healthy` for app containers.
- `erp_schema.sql` loads automatically on first startup.

9. Verification Checklist

- **Check 1 – Single-command startup:** Clean start with `docker compose down -v` followed by `docker compose up --build -d`. All six services healthy.
- **Check 2 – End-to-end processing:** Place an invoice in the incoming folder and confirm extraction, translation, validation, and database reporting.
- **Check 3 – Persistence across restart:** Process an invoice, run `docker compose down` then `docker compose up -d`, and verify audit records and RAG queries still work.
- **Check 4 – Deliberate data loss:** Run `docker compose down -v` then `docker compose up -d` to confirm volume wiping resets data.
- **Check 5 – No secrets in images:** Verified via `docker history <image> --no-trunc`.
- **Check 6 – Service-name resolution:** Verify container network DNS by resolving hostnames inside `agent`.

10. Deliverables & Evaluation Criteria

Deliverables

- Source code (3 Dockerfiles, 3 .dockerignore files, compose.yaml).
- Audit schema SQL definition.
- README.md with configuration instructions.
- Working demonstration covering the 6 verification checks.
- Presentation slides (≤ 5 slides) with deployed architecture diagram.
- Prompt templates and persona files (.yaml/.json).

Evaluation Breakdown (50 Marks Total)

- **Functional completeness:** 25%
- **RAG and agentic integration:** 25%
- **Containerization and deployment:** 25%
  - Full stack runs from a single command: 10%
  - Dockerfile quality (non-root, layer cache, no secrets): 10%
  - Persistence correctly configured: 5%
- **Code quality and modularity:** 15%
- **UI and human-in-the-loop:** 10%